

A New Approach For Compiling C++

Document #: P2978R0
Date: 2023-09-22
Project: Programming Language C++
Audience: SG15
Reply-to: HassanSajjad
<hassan.sajjad069@gmail.com>

Contents

- [1 Abstract](#)
- [2 Changes](#)
 - [2.1 R0 \(Initial\)](#)
- [3 Introduction](#)
- [4 Motivation and Scope](#)
 - [4.1 Why is this faster](#)
 - [4.2 How much faster this could be?](#)
- [5 Impact On the Standard](#)
- [6 Design Decisions](#)
 - [6.1 What are the tradeoffs?](#)
- [7 Technical Specifications](#)

§ 1 Abstract

This paper specifies the API for the interaction between the build system and the compiler where the compiler is available as a shared library. This could result in improved compilation speed.

§ 2 Changes

§ 2.1 R0 (Initial)

§ 3 Introduction

Today, almost all compilation happens by the user or build system invoking the compiler executable. This paper proposes the availability of the compiler as a shared library. Both the compiler executable and the compiler shared library can co-exist. The build system can then interact with the shared library with the API specified in this paper. Compared to the current approach, this will result in faster compilation speed, close to 25 - 40 % in some cases.

§ 4 Motivation and Scope

Most operating systems today support dynamic loading of the shared library.

§ 4.1 Why is this faster

- i. API allows the build system to intercept the files read by the compiler. So, the compilation of multiple files can use the same cached file.
- ii. Module files need to be scanned to determine the dependencies of the file. Only after that, the files could be compiled in order. In this approach, the scanning is not needed.

§ 4.2 How much faster this could be?

Tests conducted imitating real-world scenarios revealed that it could be 25 - 40 % faster <https://github.com/HassanSajjad-302/solution5>.

Tests were performed with the C++20 MSVC compiler on the Windows 11 operating system on modern hardware at the time of writing. Repositories used include SFML and LLVM.

The highlights include.

- i. The estimated scanning time percent of the total compilation time (scanning time + compilation time) per file for LLVM is 8.46%.
- ii. SFML was compiled with C++20 header units. Because compilation became faster, the scanning time took a larger proportion of the total time. Scanning took 25.72% of the total compilation time. For a few files scanning was slower than compilation.
- iii. The estimated average scanning time per file for the project LLVM was 217.9ms. For some files, it was more than 400ms.
- iv. On average the LLVM source file includes 400 header files. 3598 unique header files are read from the disk while compiling 2854 source files. If LLVM is built with C++20 modules or C++20 header units, there will be 2854 + 3598 process launches instead of 2854. More processes are needed for the compilation of module interface files or header units. Few compilers use two phase model instead of one phase.

The distinction between the two models is discussed here: https://gitlab.kitware.com/cmake/cmake/-/issues/18355#note_1329192. In such a case, there will be $2854 + (2 * 3598)$ process launches instead of 2854. Avoiding these costs of process setup and file reads should result in a 1 - 2 % compilation speed-up in a clean build in the project size of LLVM.

§ 5 Impact On the Standard

This proposal does not impact the current build systems in any way. Few build systems, however, might need non-trivial changes to support this. Plans for supporting this in the build system HMake are explained here: <https://lists.isocpp.org/sg15/att-2033/Analysis.pdf>

§ 6 Design Decisions

§ 6.1 What are the tradeoffs?

1. Memory consumption of such an approach could be higher than the current approach. That is because more `compiler_state` and the cached files need to be kept in the memory.

However, this can be alleviated by the following:

In some build systems today, related files are grouped as one target. A target can depend on one or more other target. Such build systems can alleviate the higher memory consumption by ordering the compilation of files of the dependency before the dependent target. This way only the `compile_state` of the files of the dependency target need to be kept in the memory. `compiler_state` of the files of the dependent target only comes into the picture once the dependent target files are compiled. BMI files of a target are kept in the memory until there is no file of any dependent target left to be compiled. At which point, this is cleared from the memory.

In case a similar file is being read by multiple compilations, the memory consumption could be a little less than the current approach as all such compilations can use one cached read instead of reading themselves.

2. The build system can not handle the compiler crash. This may lead to build process termination. But build system can support both models, so the user has the option to fall back.
3. In some cases, modifications to the configuration controlling the resource limitations of the build process might be needed as well.
4. While the administrator cannot diagnose a hung or long-running process from the process listing, the build system can detect it by registering thread IDs and

timestamps before the “newCompile” and “resumeCompile” calls, and it can alert the user if the thread ID is not cleared soon.

§ 7 Technical Specifications

Compilation pause and resume capability needs to be built into the compiler. The compiler shared library must be able to do multiple compilations in one process concurrently.

```
namespace buildsystem
{
    struct string
    {
        const char *ptr;
        unsigned long size;
    };

    // Those char pointers that are pointing to the path are platform
    // dependent i.e. whcar_t* in-case of Windows
    struct compile_output
    {
        // if (!completed), then pointer to the compiler state to be preserved
        // by the build system, else nullptr
        void *compiler_state;

        // if (completed), then compiler output and errorOutput, else nullptr
        string stdout;
        string stderr;

        // if (!completed), then one of module name or header unit path of the
        // module or header unit the compiler is waiting
        // on, else if(completed), then the logical_name of exported module if
        // any.
        string wait_string_or_logical_name;

        // Following is the array size, next is the array of header includes.
        unsigned long header_includes_count;
        string *header_includes;

        // Following is the array of files returned, next is the array of
        // filesystem paths of these files, next is the size
        // of these arrays.
        string *output_files;
        string *output_files_paths;
        unsigned short output_files_count;

        // true if compiler is waiting on module, false otherwise.
        bool waiting_on_module;

        // true if compilation completes or an error occurs, false otherwise.
        bool completed;

        // if (completed), then true if an error occurred, false otherwise.
        bool error_occurred;
    };
}
```

```
};

compile_output      new_compile(string      compile_command,      string
                               (*get_file_contents)(string file_path));
compile_output resume_compile(void *compiler_state, string bmi_file);
string get_object_file(string bmi_file);

} // namespace buildsystem
```

The compiler calls `new_compile` function passing it the `compile_command` for the module file. The compile command, however, does not include any dependencies. If the compiler sees an import of a module, it sets the `wait_string_or_logical_name` string of the `compile_output` return value to the name of the module. It also sets `waiting_on_module` to true. If the compiler sees an import of a header unit, it sets the `wait_string_or_logical_name` string of the `compile_output` return value to the path of the header unit. It also sets `waiting_on_module` to false.

The build system now will preserve the `compiler_state` and will check if the required file is already built, or it needs to be built, or it is being built. Only after the file is available, the build system will call `resume_compile` function passing it the BMI file. `resume_compile` is called until the file has no dependency not provided and the compilation completes. The compiler returns the files in array `output_files` and their corresponding output paths in the array `output_files_paths`. From that path build system can establish the type of file.

If the compiler uses two phase model, only the BMI file is returned, and the build system can later call `get_object_file` to get the object file. The argument `get_file_contents` is used by the compiler to get the contents of any file instead of reading itself. This means that a file does not get read twice for different compilations. As compilation completes, the build system will write BMI and object files to the disk as well.